Modul Hari 5 - Fungsi, Array, DOM, dan Modularisasi (Proyek 2)

Modul ini memuat teori dan praktik Hari 5 diklat Junior Web Programmer (lanjutan unit J.620100.017.02 Mengimplementasikan Pemrograman Terstruktur), dengan jadwal 08.00-17.00 = 10 JP (1 JP = 45 menit). Hasil akhirnya adalah Proyek 2: Kalkulator dan Form Validator.

Tujuan Pembelajaran

1. Membuat program dengan prosedur dan fungsi (modularisasi), termasuk parameter, nilai kembalian, dan cakupan variabel.
2. Mengolah data dengan array dan method array (`map`, `filter`, `reduce`, `find`, `sort`).
3. Memecah program menjadi beberapa berkas modul JavaScript.
4. Memanipulasi DOM dan menangani event untuk membangun antarmuka interaktif.
5. Memvalidasi input form di sisi klien dengan pesan kesalahan yang jelas dan aksesibel.
6. Menangani kasus tepi dan kesalahan dengan `try...catch`.

Jadwal

Waktu	Kegiatan
08.00-08.45	Review Pengolah Nilai dan kuis, Teori T1: fungsi dan scope
08.45-10.15	Teori T2-T7: array, objek, modularisasi, DOM, event, validasi form, penanganan error
10.30-12.00	Lab 1: refactor Pengolah Nilai ke fungsi; Lab 2: array dan method array
13.00-14.30	Lab 3: DOM dan event; Lab 4: modul ES + Bug Hunt
14.45-16.15	PBL: Proyek 2 - Kalkulator dan Form Validator
16.15-17.00	Code review, commit, refleksi

Materi Teori

T1. Fungsi dan Cakupan (Scope)

Fungsi adalah blok kode bernama yang dapat dipanggil berulang kali. Fungsi yang mengembalikan nilai disebut *fungsi*; yang hanya menjalankan aksi tanpa mengembalikan nilai disebut *prosedur*. Prinsip: **satu fungsi, satu tanggung jawab**; nama berupa kata kerja (`hitungRataRata`, `tampilkanMenu`).

```
// Deklarasi fungsi
function hitungLuasPersegi(sisi) {
  return sisi * sisi;
}

// Function expression dan arrow function
const hitungKeliling = function (sisi) { return 4 * sisi; };
```

```
const hitungDiagonal = (sisi) => sisi * Math.SQRT2;

// Parameter default
function sapa(nama = 'Tamu') {
  return `Halo, ${nama}!`;
}

// Prosedur (tanpa return)
function cetakGaris() {
  console.log('-'.repeat(20));
}

// Fungsi dengan banyak nilai kembalian (objek)
function statistik(daftar) {
  return {
    jumlah: daftar.length,
    tertinggi: Math.max(...daftar),
    terendah: Math.min(...daftar)
  };
}
```

Istilah	Arti
Parameter	Variabel di definisi fungsi (`function f(a, b)`)
Argumen	Nilai yang dikirim saat memanggil (`f(2, 3)`)
`return`	Mengembalikan nilai dan menghentikan fungsi
Fungsi murni	Hasil hanya bergantung pada argumen, tanpa mengubah data di luar (mudah diuji)
Callback	Fungsi yang dikirim sebagai argumen ke fungsi lain

Scope (cakupan variabel):

Jenis	Cakupan
Global	Seluruh program; hindari menaruh banyak variabel di sini
Fungsi	Hanya di dalam fungsi tempat dideklarasikan
Blok	`let` dan `const` hanya hidup di dalam `{ }` terdekat

```
const pajak = 0.11;                 // global (konstanta)
function hitungTotal(harga) {
  const total = harga * (1 + pajak); // lokal
  return total;
}
// console.log(total);              // ReferenceError: total tidak terlihat di luar
fungsi
```

Dokumentasi singkat fungsi (JSDoc):

```
/**
 * Menghitung rata-rata dari daftar nilai.
 * @param {number[]} daftarNilai - Array angka.
 * @returns {number} Rata-rata, atau 0 bila array kosong.
 */
function hitungRataRata(daftarNilai) {
  if (daftarNilai.length === 0) return 0;
  const jumlah = daftarNilai.reduce((total, nilai) => total + nilai, 0);
  return jumlah / daftarNilai.length;
}
```

T2. Array dan Method Array

```
const buah = ['apel', 'jeruk', 'mangga'];
buah[0];                 // 'apel' (indeks dimulai dari 0)
buah.length;             // 3
buah.push('pisang');     // tambah di akhir
buah.pop();              // hapus dari akhir
buah.unshift('anggur');  // tambah di awal
buah.shift();            // hapus dari awal
buah.indexOf('jeruk');   // 1 (-1 bila tidak ada)
buah.includes('apel');   // true
buah.slice(0, 2);        // salinan elemen indeks 0-1
buah.splice(1, 1);       // hapus 1 elemen mulai indeks 1 (mengubah array)
buah.join(', ');         // 'apel, mangga'
```

Method	Fungsi	Mengubah array asli?
`forEach(fn)`	Menjalankan fungsi untuk tiap elemen	Tidak
`map(fn)`	Membuat array baru hasil transformasi	Tidak
`filter(fn)`	Membuat array baru berisi elemen yang lolos	Tidak
`reduce(fn, awal)`	Menggabungkan menjadi satu nilai	Tidak
`find(fn)`	Elemen pertama yang cocok (atau `undefined`)	Tidak
`some(fn)` / `every(fn)`	Apakah ada / semua elemen yang cocok	Tidak
`sort(fn)`	Mengurutkan (beri fungsi pembanding untuk angka)	**Ya**
`reverse()`	Membalik urutan	**Ya**

goodigitaledu

```
const nilai = [80, 65, 90, 72];

const lulus = nilai.filter((n) => n >= 70);          // [80, 90, 72]
const naik5 = nilai.map((n) => n + 5);               // [85, 70, 95, 77]
const total = nilai.reduce((jumlah, n) => jumlah + n, 0); // 307
const pertama = nilai.find((n) => n > 85);           // 90
const adaGagal = nilai.some((n) => n < 70);          // true
const urutNaik = [...nilai].sort((a, b) => a - b);   // salinan terurut

// Destructuring dan spread
const [pertamaNilai, keduaNilai] = nilai;
const gabungan = [...nilai, 100];
```

Array dua dimensi (matriks): `const papan = [[1, 2], [3, 4]]; papan[1][0]; // 3`.

T3. Array of Object (Data Berstruktur)

Data nyata biasanya berupa daftar objek.

```
const peserta = [
  { id: 1, nama: 'Sari', kelas: 'Web', nilai: 90 },
  { id: 2, nama: 'Budi', kelas: 'Web', nilai: 68 },
  { id: 3, nama: 'Citra', kelas: 'Data', nilai: 82 }
];

const namaWeb = peserta.filter((p) => p.kelas === 'Web').map((p) => p.nama);
const rata = peserta.reduce((j, p) => j + p.nilai, 0) / peserta.length;
const budi = peserta.find((p) => p.nama === 'Budi');
const urutNilai = [...peserta].sort((a, b) => b.nilai - a.nilai);

// Menghitung jumlah per kelas
const perKelas = {};
for (const p of peserta) {
  perKelas[p.kelas] = (perKelas[p.kelas] || 0) + 1;
}
```

T4. Modularisasi Program

Membagi program menjadi bagian kecil yang terpisah mempermudah pembacaan, pengujian, dan pemakaian ulang. Prinsip pemisahan: **data/logika** terpisah dari **tampilan (UI)**.

```
proyek-2-kalkulator/
  index.html
  css/style.css
  js/
    operasi.js      # logika perhitungan (fungsi murni)
    app.js          # penghubung DOM dan event
```

ES Module memakai `export` dan `import`:

```
// js/operasi.js
export const tambah = (a, b) => a + b;
export function bagi(a, b) {
  if (b === 0) throw new Error('Tidak bisa dibagi nol');
  return a / b;
}

// js/app.js
import { tambah, bagi } from './operasi.js';
```

```
<script type='module' src='js/app.js'></script>
```

Catatan: modul ES hanya berjalan lewat server (`http://`), bukan `file:///`. Gunakan ekstensi **Live Server** atau `htdocs` XAMPP.

T5. DOM dan Event

DOM (Document Object Model) adalah representasi HTML sebagai pohon objek yang dapat dibaca dan diubah JavaScript.

```
document
  html
    body
      h1#judul
      ul#daftar
        li
        li
```

Tujuan	Kode
Memilih satu elemen	`document.getElementById('judul')`, `document.querySelector('.kartu')`
Memilih banyak elemen	`document.querySelectorAll('li')`
Mengubah teks	`el.textContent = 'Halo'`
Membaca/mengisi input	`input.value`
Mengatur class	`el.classList.add('aktif')`, `remove`, `toggle`

Tujuan	Kode
Mengatur atribut	`el.setAttribute('aria-invalid', 'true')`
Membuat elemen	`const li = document.createElement('li');` `daftar.append(li);`
Menghapus elemen	`li.remove()`

Keamanan: gunakan `textContent` untuk menampilkan data dari pengguna. Hindari `innerHTML` dengan data pengguna karena membuka celah XSS (penyisipan skrip berbahaya).

```
const tombol = document.querySelector('#tombol');

tombol.addEventListener('click', (event) => {
  console.log('Diklik', event.target);
});
```

Event	Terjadi saat
`click`	Elemen diklik
`input`	Isi input berubah (tiap ketukan)
`change`	Nilai input selesai diubah
`submit`	Form dikirim (gunakan `event.preventDefault()` untuk mencegah muat ulang)
`keydown`	Tombol keyboard ditekan
`DOMContentLoaded`	Dokumen selesai dimuat

Event delegation: pasang satu pendengar pada induk (mis. daftar atau papan tombol) dan periksa `event.target.closest('button')`. Cara ini efisien untuk elemen dinamis.

goodigitaledu

T6. Validasi Form Sisi Klien

Validasi klien meningkatkan pengalaman pengguna, tetapi **tidak aman**: pengguna dapat melewatinya, jadi server tetap wajib memvalidasi ulang (Hari 8).

Aturan	Ekspresi
Panjang minimum	`nilai.trim().length >= 3`
Email	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(nilai)`
Nomor HP Indonesia	`/^08[0-9]{8,11}$/.test(nilai)`
Sandi kuat	panjang >= 8, ada huruf dan angka
Konfirmasi sandi	sama dengan kolom sandi

Praktik terbaik: tampilkan pesan di bawah kolom terkait, tandai dengan `aria-invalid`, beri `aria-live='polite'` pada wadah pesan, validasi saat `input` dan saat `submit`, dan jangan hanya mengandalkan warna merah.

T7. Penanganan Kesalahan dan Kasus Tepi

```
function bagi(a, b) {
  if (b === 0) throw new Error('Tidak bisa dibagi nol');
  return a / b;
}

try {
  console.log(bagi(10, 0));
} catch (error) {
  console.error('Gagal: ' + error.message);
} finally {
  console.log('Selesai');
}
```

Daftar kasus tepi yang wajib diuji: input kosong, hanya spasi, nol, negatif, angka sangat besar, teks pada kolom angka, dan array kosong.

Praktik Hari 5

Lab 1 - Refactor Pengolah Nilai ke Fungsi

Tujuan: memecah Pengolah Nilai (Hari 4) menjadi fungsi murni yang dapat diuji.

1. Salin `nilai.js` Hari 4 ke `hari-05-fungsi/nilai-fungsi.js`.
2. Buat fungsi berikut (tanpa membaca variabel global langsung):

```
const BATAS_LULUS = 70;

function hitungRataRata(daftar) {
  if (daftar.length === 0) return 0;
  return daftar.reduce((jumlah, n) => jumlah + n, 0) / daftar.length;
}

function cariTertinggi(daftar) {
  return daftar.length === 0 ? null : Math.max(...daftar);
}

function cariTerendah(daftar) {
  return daftar.length === 0 ? null : Math.min(...daftar);
}

function tentukanGrade(nilai) {
  if (nilai >= 85) return 'A';
  if (nilai >= 70) return 'B';
  if (nilai >= 55) return 'C';
  return 'D';
}

function tentukanStatus(nilai) {
  return nilai >= BATAS_LULUS ? 'Lulus' : 'Tidak lulus';
}

function nilaiValid(nilai) {
  return Number.isFinite(nilai) && nilai >= 0 && nilai <= 100;
}
```

3. Buat pengujian sederhana dengan `console.assert` (tidak mencetak apa pun bila benar):

```
console.assert(hitungRataRata([80, 90]) === 85, 'rata-rata 80,90 harus 85');
console.assert(hitungRataRata([]) === 0, 'rata-rata array kosong harus 0');
console.assert(tentukanGrade(85) === 'A', 'nilai 85 harus A');
console.assert(tentukanGrade(69) === 'C', 'nilai 69 harus C');
console.assert(nilaiValid(101) === false, '101 tidak valid');
```

4. Tambahkan fungsi prosedur `tampilkanMenu()` dan `tampilkanLaporan(nama, nilai)`; panggil semuanya dari fungsi `main()`.
5. Tambahkan JSDoc pada setiap fungsi.

Lab 2 - Array dan Method Array

Gunakan data `peserta` pada T3 (tambahkan minimal 8 peserta). Tulis fungsi untuk:

1. Daftar nama peserta yang lulus (`filter` dan `map`).
2. Rata-rata nilai per kelas.
3. Peserta dengan nilai tertinggi (`reduce` atau `sort`).
4. Pencarian peserta berdasarkan sebagian nama (`filter` dan `includes`, tidak peka huruf besar-kecil).
5. Daftar peringkat (`sort` pada salinan array) dengan nomor urut.
6. Menghapus duplikat dari array email memakai `new Set(...)`.

Lab 3 - DOM dan Event

Buat counter dan daftar tugas (to-do) sederhana.

```
<h2 id='angka'>0</h2>
<button id='tambah'>Tambah</button>
<button id='kurang'>Kurang</button>
<button id='reset'>Reset</button>

<form id='form-tugas'>
  <label for='teks-tugas'>Tugas baru</label>
  <input id='teks-tugas' required>
  <button type='submit'>Tambah</button>
</form>
<ul id='daftar-tugas'></ul>

<script>
  let hitung = 0;
  const angka = document.getElementById('angka');
  const perbarui = () => { angka.textContent = hitung; };

  document.getElementById('tambah').addEventListener('click', () => { hitung++;
perbarui(); });
  document.getElementById('kurang').addEventListener('click', () => { hitung--;
perbarui(); });
  document.getElementById('reset').addEventListener('click', () => { hitung = 0;
perbarui(); });

  const form = document.getElementById('form-tugas');
  const daftar = document.getElementById('daftar-tugas');

  form.addEventListener('submit', (event) => {
    event.preventDefault();
    const teks = document.getElementById('teks-tugas').value.trim();
    if (teks === '') return;

    const item = document.createElement('li');
    item.textContent = teks + ' ';
    const hapus = document.createElement('button');
    hapus.type = 'button';
```

```
    hapus.textContent = 'Hapus';
    hapus.addEventListener('click', () => item.remove());
    item.append(hapus);
    daftar.append(item);
    form.reset();
  });
</script>
```

Tantangan: tandai tugas selesai dengan `classList.toggle('selesai')` (coret teks lewat CSS), tampilkan jumlah tugas tersisa, dan batasi teks maksimal 80 karakter.

Lab 4 - Modul ES

1. Buat folder `hari-05-modul/` berisi `index.html`, `js/matematika.js`, `js/app.js`.
2. Pindahkan `hitungRataRata`, `cariTertinggi`, dan `cariTerendah` ke `matematika.js` dengan `export`.
3. Impor di `app.js`, tampilkan hasilnya pada elemen halaman dari array nilai di dalam `<input>` yang dipisahkan koma.
4. Muat dengan `<script type='module' src='js/app.js'></script>` lewat Live Server. Amati pesan error bila dibuka dengan `file:///` dan jelaskan penyebabnya.

Bug Hunt Hari 5

Instruktur membagikan `bug-fungsi.js` dan `bug-dom.html` berisi 8 kesalahan.

No	Kode bermasalah	Gejala	Perbaikan
1	Fungsi tanpa `return` lalu hasil dipakai	Hasil `undefined`	Tambahkan `return`
2	`hitungRataRata(daftar)` tidak memeriksa array kosong	Hasil `NaN`	Tangani kasus kosong
3	`document.getElementById('Tombol')` padahal `id='tombol'`	`null`, `TypeError` saat `addEventListener`	Samakan huruf besar-kecil
4	Script di `head` tanpa `defer`	Elemen belum ada saat dicari	Pindah ke akhir `body` atau pakai `defer`
5	`form` tidak memanggil `event.preventDefault()`	Halaman termuat ulang	Tambahkan `preventDefault()`
6	`daftar.sort()` pada angka	Urutan salah (`[10, 2, 5]`)	`sort((a, b) => a - b)`

No	Kode bermasalah	Gejala	Perbaikan
7	`for (var i...)` dengan penutup (closure) di dalam loop	Semua tombol memakai `i` terakhir	Gunakan `let`
8	`el.innerHTML = inputPengguna`	Celah XSS	Gunakan `textContent`

PBL Hari 5 - Proyek 2: Kalkulator dan Form Validator

Proyek memakai struktur folder berikut:

```
proyek-2-kalkulator/
  kalkulator/
    index.html
    css/style.css
    js/operasi.js
    js/app.js
  validator/
    index.html
    css/style.css
    js/aturan.js
    js/app.js
  README.md
```

Bagian A - Kalkulator

Spesifikasi: tombol angka 0-9 dan titik desimal, operator tambah, kurang, kali, bagi, tombol C (hapus) dan = (sama dengan); layar tampil di atas; pembagian dengan nol menampilkan pesan, bukan `Infinity`; tombol disusun dengan CSS Grid (Hari 3); logika terpisah dari UI.

`js/operasi.js`

```
export const tambah = (a, b) => a + b;
export const kurang = (a, b) => a - b;
export const kali = (a, b) => a * b;

export function bagi(a, b) {
  if (b === 0) throw new Error('Tidak bisa dibagi nol');
  return a / b;
}

export function hitung(operator, a, b) {
  switch (operator) {
    case '+': return tambah(a, b);
    case '-': return kurang(a, b);
    case '*': return kali(a, b);
    case '/': return bagi(a, b);
    default: throw new Error('Operator tidak dikenal');
  }
}
```

`index.html` (bagian utama)

```
<main class='kalkulator'>
  <div id='layar' class='layar' aria-live='polite'>0</div>
  <div id='papan' class='papan'>
    <button data-aksi='hapus'>C</button>
    <button data-operator='/'>/</button>
    <button data-operator='*'>x</button>
    <button data-operator='-'>-</button>
    <button data-angka='7'>7</button>
    <button data-angka='8'>8</button>
    <button data-angka='9'>9</button>
    <button data-operator='+'>+</button>
    <button data-angka='4'>4</button>
    <button data-angka='5'>5</button>
    <button data-angka='6'>6</button>
    <button data-aksi='sama' class='sama'>=</button>
    <button data-angka='1'>1</button>
    <button data-angka='2'>2</button>
    <button data-angka='3'>3</button>
    <button data-angka='0' class='nol'>0</button>
    <button data-angka='.'>.</button>
  </div>
</main>
<script type='module' src='js/app.js'></script>
```

`css/style.css`

```
* { box-sizing: border-box; }
body { font-family: system-ui, sans-serif; display: grid; place-items: center;
min-height: 100vh; margin: 0; background: #f3f4f6; }
.kalkulator { width: min(320px, 92vw); padding: 1rem; background: #fff;
border-radius: 12px; box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12); }
.layar { min-height: 3.5rem; padding: 0.75rem; margin-bottom: 0.75rem; text-align:
right; font-size: 1.75rem; background: #111827; color: #fff; border-radius: 8px;
overflow-x: auto; }
.papan { display: grid; grid-template-columns: repeat(4, 1fr); gap: 0.5rem; }
.papan button { padding: 0.9rem 0; font-size: 1.1rem; border: 0; border-radius: 8px;
background: #e5e7eb; cursor: pointer; }
.papan button:hover { background: #d1d5db; }
.papan .sama { grid-row: span 2; background: #0d6efd; color: #fff; }
.papan .nol { grid-column: span 2; }
```

`js/app.js`

```
import { hitung } from './operasi.js';

const layarEl = document.getElementById('layar');
const papan = document.getElementById('papan');

let layar = '0';
let angkaPertama = null;
let operator = null;
let siapReset = false;

function perbaruiLayar() {
```

```
  layarEl.textContent = layar;
}

function masukkanAngka(digit) {
  if (digit === '.' && layar.includes('.') && !siapReset) return;
  if (layar === '0' && digit !== '.') {
    layar = digit;
  } else if (siapReset) {
    layar = digit === '.' ? '0.' : digit;
    siapReset = false;
  } else {
    layar += digit;
  }
}

function hitungHasil() {
  if (operator === null) return;
  try {
    const hasil = hitung(operator, angkaPertama, Number(layar));
    layar = String(Number(hasil.toFixed(10)));
  } catch (error) {
    layar = error.message;
  }
  operator = null;
  angkaPertama = null;
  siapReset = true;
}

function pilihOperator(op) {
  if (operator !== null && !siapReset) hitungHasil();
  angkaPertama = Number(layar);
  operator = op;
  siapReset = true;
}

function bersihkan() {
  layar = '0';
  angkaPertama = null;
  operator = null;
  siapReset = false;
}

papan.addEventListener('click', (event) => {
  const tombol = event.target.closest('button');
  if (!tombol) return;

  if (tombol.dataset.angka !== undefined) masukkanAngka(tombol.dataset.angka);
  else if (tombol.dataset.operator) pilihOperator(tombol.dataset.operator);
  else if (tombol.dataset.aksi === 'sama') hitungHasil();
  else if (tombol.dataset.aksi === 'hapus') bersihkan();

  perbaruiLayar();
});
```

Kasus tepi yang harus lolos: `0.1 + 0.2` tampil 0,3; `5 / 0` menampilkan pesan lalu `C` memulihkan; menekan operator dua kali; angka diawali titik; hasil panjang tidak merusak layout.

Tantangan: dukungan keyboard (`keydown`), tombol hapus satu karakter, riwayat 5 perhitungan terakhir dalam array.

Bagian B - Form Validator

Spesifikasi: form pendaftaran akun dengan kolom nama, email, nomor HP, kata sandi, dan konfirmasi sandi; validasi saat mengetik dan saat dikirim; pesan di bawah kolom; tombol Kirim nonaktif sampai semua valid; setelah valid tampil ringkasan data (tanpa sandi) dengan `textContent`.

`js/aturan.js`

```
export function validasiNama(nilai) {
  return nilai.trim().length >= 3 ? '' : 'Nama minimal 3 karakter.';
}

export function validasiEmail(nilai) {
  const pola = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return pola.test(nilai) ? '' : 'Format email tidak valid.';
}

export function validasiHp(nilai) {
  return /^08[0-9]{8,11}$/.test(nilai) ? '' : 'Nomor HP diawali 08 dan 10-13 digit.';
}

export function validasiSandi(nilai) {
  const kuat = nilai.length >= 8 && /[0-9]/.test(nilai) && /[A-Za-z]/.test(nilai);
  return kuat ? '' : 'Sandi minimal 8 karakter, kombinasi huruf dan angka.';
}

export function validasiKonfirmasi(nilai, sandi) {
  return nilai === sandi ? '' : 'Konfirmasi sandi tidak cocok.';
}
```

`index.html` (satu kolom sebagai pola; ulangi untuk kolom lain)

```
<form id='form-daftar' novalidate>
  <p>
    <label for='nama'>Nama</label>
    <input id='nama' name='nama' autocomplete='name' aria-describedby='galat-nama'>
    <span id='galat-nama' class='galat' aria-live='polite'></span>
  </p>
  <!-- ulangi: email, hp, sandi (type='password'), konfirmasi (type='password') -->
  <button id='kirim' type='submit' disabled>Kirim</button>
</form>
<section id='ringkasan' hidden></section>
<script type='module' src='js/app.js'></script>
```

`js/app.js`

```
import {
  validasiNama, validasiEmail, validasiHp, validasiSandi, validasiKonfirmasi
} from './aturan.js';
```

```
const form = document.getElementById('form-daftar');
const tombolKirim = document.getElementById('kirim');
const ringkasan = document.getElementById('ringkasan');

const aturan = {
  nama: (nilai) => validasiNama(nilai),
  email: (nilai) => validasiEmail(nilai),
  hp: (nilai) => validasiHp(nilai),
  sandi: (nilai) => validasiSandi(nilai),
  konfirmasi: (nilai) => validasiKonfirmasi(nilai, form.elements.sandi.value)
};

function tampilkanGalat(nama, pesan) {
  document.getElementById(`galat-${nama}`).textContent = pesan;
  form.elements[nama].setAttribute('aria-invalid', pesan ? 'true' : 'false');
}

function periksaField(nama) {
  const pesan = aturan[nama](form.elements[nama].value);
  tampilkanGalat(nama, pesan);
  return pesan === '';
}

function semuaValid() {
  return Object.keys(aturan).every((nama) => aturan[nama](form.elements[nama].value)
=== '');
}

form.addEventListener('input', (event) => {
  const nama = event.target.name;
  if (nama in aturan) periksaField(nama);
  if (nama === 'sandi' && form.elements.konfirmasi.value !== '')
periksaField('konfirmasi');
  tombolKirim.disabled = !semuaValid();
});

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const hasil = Object.keys(aturan).map(periksaField);
  if (!hasil.every(Boolean)) return;

  ringkasan.hidden = false;
  ringkasan.textContent = `Pendaftaran berhasil: ${form.elements.nama.value}
(${form.elements.email.value})`;
  form.reset();
  tombolKirim.disabled = true;
});
```

Rubrik Proyek 2 (100 poin):

Aspek	Bobot	Indikator
Fungsionalitas	30	Kalkulator dan validator berjalan sesuai spesifikasi
Modularisasi fungsi dan berkas	25	Logika terpisah dari UI, fungsi bertanggung jawab tunggal
Penanganan kasus tepi	20	Nol, kosong, input salah, urutan operator ganda
Kebersihan kode	15	Penamaan, indentasi, JSDoc, tanpa duplikasi, tanpa `onclick` inline
README dan Git	10	Cara pakai, tangkapan layar, commit bermakna

Code Review dan Commit

Butir	Ya/Tidak	Catatan
Tidak ada fungsi lebih dari sekitar 20 baris		
Tidak ada duplikasi kode (DRY)		
Event dipasang dengan `addEventListener`, bukan atribut inline		
Data pengguna ditampilkan dengan `textContent`		
Kasus tepi diuji dan terdokumentasi		
Fungsi publik memiliki JSDoc		

```
git add .
git commit -m 'Hari 5: Proyek 2 kalkulator dan form validator'
git push
```

Bukti Portofolio Hari 5

1. `nilai-fungsi.js` dengan pengujian `console.assert`.
2. Folder `proyek-2-kalkulator/` (dua aplikasi) dan README berisi tangkapan layar.
3. Tabel Bug Hunt dan daftar kasus tepi yang diuji.

Evaluasi Hari 5

Kuis tertulis (10 soal):

1. Fungsi yang tidak memakai `return` mengembalikan ... (A. 0; B. `null`; C. `undefined`; D. string kosong).
2. Variabel `let` yang dideklarasikan di dalam `{ }` dapat dipakai ... (A. di seluruh program; B. hanya di blok itu; C. di fungsi lain; D. di berkas lain).
3. Hasil `[1, 2, 3].map((n) => n * 2)` adalah ... (A. `[2, 4, 6]`; B. `12`; C. `[1, 2, 3]`; D. `undefined`).
4. Method untuk menjumlahkan semua elemen menjadi satu nilai adalah ... (A. `map`; B. `filter`; C. `reduce`; D. `find`).
5. `[10, 2, 5].sort()` tanpa pembanding menghasilkan ... (A. `[2, 5, 10]`; B. `[10, 2, 5]`; C. urutan string `[10, 2, 5]`; D. error).
6. Cara menampilkan data dari pengguna yang aman adalah ... (A. `innerHTML`; B. `textContent`; C. `eval`; D. `document.write`).
7. Untuk mencegah form memuat ulang halaman dipakai ... (A. `event.stop()`; B. `event.preventDefault()`; C. `return true`; D. `form.cancel()`).
8. Modul ES hanya berjalan melalui ... (A. `file:///`; B. server HTTP; C. terminal saja; D. Photoshop).
9. Keuntungan event delegation adalah ... (A. memakai banyak pendengar; B. satu pendengar pada induk melayani banyak anak; C. mematikan event; D. mempercepat CSS).
10. Mengapa validasi sisi klien tidak cukup? (A. Browser tidak mendukung; B. Dapat dilewati sehingga server harus memvalidasi ulang; C. Hanya untuk angka; D. Membuat halaman lambat).

Kunci: 1-C, 2-B, 3-A, 4-C, 5-C, 6-B, 7-B, 8-B, 9-B, 10-B.

Ceklis keluaran:

- ☐ Pengolah Nilai versi fungsi lolos pengujian `console.assert`.
- ☐ Kalkulator menangani pembagian dengan nol dan kasus tepi.
- ☐ Form Validator menampilkan pesan dan menonaktifkan tombol dengan benar.
- ☐ Kode modular (berkas terpisah) dan lolos code review.
- ☐ Commit dan push selesai.

goodigitaledu